

Javascript

Page No.

Date

* Variables:

①

`var name = "ABC";`

- (1) old way of declaring variables.
- (2) can be re-declared & updated.
- (3) Function-scoped

②

`let age = 25;`

- (1) Block-scoped (only exists inside `{ }`)
- (2) can be updated, but not re-declared in same scope.

③

`const pi = 3.14;`

- (1) Block-scoped.
- (2) cannot be updated or re-declared.
- (3) Must be initialized when declared.

Uninitialized variables becomes undefined in JS:

* `let accountState;` $\longrightarrow$ Value: undefined (not null)

*

Block Scope Vs function-scope:

*

Block Scope: Variables declared exist inside a specific block only

E.x. `let a = 10`

`const b = 20;`

`console.log(a);`

`console.log(b);`

} Variables are declared in the global scope of the JS file.

`function () {`

`let x = 5;`

`.....`

`}`

} x is limited only to this block.

`console.log(x)` $\longrightarrow$ Error

* "use strict"; → treat all JS code as newer version

↑
written at the top of a JS file.

Page No.	
Date	

* function scope: Variable declared are accessible anywhere inside the function where they are declared.

Var ignores blocks & only cares about functions.

① let

```
function demo() {  
  if (true) {  
    let a = 5;  
  }  
  console.log(a) → Error
```

} a is in
this
block
scope

* var can accidentally modify values outside blocks, causing bugs.

② var

```
function demo() {  
  if (true) {  
    var b = 5;  
  }  
  console.log(b); → 5
```

} b survives because it
belongs to the whole function,
not just if block

That's why modern JS mostly uses let & const instead of var.

* Datatypes:

① Primitive Datatypes: stores single values.

Number → Integers, decimals.

→ JavaScript uses 64-bit floating point numbers

→ safe integer value: $-(2^{53} - 1)$ to $(2^{53} - 1)$

BigInt → used for very large integers beyond safe Number range.

String → stores text.

→ No fixed limit → limited by available memory.

Undefined → variable declared but not assigned.

Null → Represents intentional empty value.

`typeof (null)` → object

`typeof (undefined)` → undefined

- * Primitives : String, numbers, ...
- * Reference (Non primitive)
→ Array, objects, Functions

"33" → 33
"33abc" → NaN
null → 0

Page No.	
Date	

*

Datatype conversion:

undefined → NaN
"abc" → NaN

```
let score = "33" // string
console.log(typeof score); // Number
let conv_to_no = Number(score)
console.log(typeof score); // number
```

Now

score = "33abc"

let conv = Number(score)

console.log(typeof score) → number (Will not give an error)

But when → console.log(score) → NaN (Not a Number)

If score = null

→ typeof score → number

→ value → 0

If score = undefined

→ type of score → number

→ value → NaN

Boolean

String

1 → true

0 → false

" " → false

"xyz" → true

123 → "123"

true → "true"

null → "null"

undefined → "undefined"

* Special cases:

① "5" + 2 → "52" | 2 + "5" → "25"

→ (+) is a string concatenation op. converts everything to string.

② "5" - 2 → 3

→ (-) only works with numbers

∴ JS converts "5" → 5

"10" + 5 - 2 ⇒ "105" - 2 ⇒ 103 (Number)

- ③ $true + 1 \rightarrow 2$
- ④ $null + 1 \rightarrow 1$ ($null = 0$)
- ⑤ $undefined + 1 \rightarrow NaN$ (Because undefined cannot become a valid number.)

* Equality Conversion:

(1) Loose Equality ($==$):

$if("5" == 5) \Rightarrow true$

↑
Performs conversion

(2) Strict Equality ($===$):

$if("5" === 5) \Rightarrow false$

* $console.log(null > 0)$

$\rightarrow false$

↑

JS converts null into 0 (Number)

$console.log(null >= 0)$

$\rightarrow true$

* $undefined == 0$

$undefined > 0$

$undefined < 0$

↑
false

* Heap & stack Memory:

(1) stack Memory: Stores $\rightarrow$ primitives values, function calls, local variables.

characteristics: Fast access, fixed size, stored in order, Uses copy by value.

$let a = 10;$

$let b = a;$

a copy of a is stored in b.

$b = 20;$

$console.log(a) \rightarrow 10$

$console.log(b) \rightarrow 20$

(2) Heap memory stores:

- * stores objects, arrays, functions.
- * These are called reference types.
- * Use by reference (dynamic allocation).

```
let obj1 = { name: "Dev" };
let obj2 = obj1;
obj2.name = "ABC"
console.log(obj1.name); → "ABC"
```

* Strings:

```
const name = "xyz"
```

(1) `console.log(`Hello ${name}`);`

(2) `const newName = new String('ABCDE');`

→ `String { 'ABCDE' }`

↓

0: A 3: D

1: B

4: E

2: C

It's not an array, it's a string object which has (key-value) pairs.

(3) `newName.length()` → 5

(4) `newName.toUpperCase()`

`console.log(newName.toUpperCase());`

↓

Doesn't change the original string
Makes a copy string & returns it.

(5) `const s = newName.substring(0, 4)`

`console.log(s)` → 'ABCD'

Start index

End index

(6) `t = "       xyz       "`

`t.trim()` → `"xyz"`

(7) `t = "ABcccccccd"`

`t.replace('c', '-')` → `"AB-----D"`

* Javascript array are resizable and can contain a mix of different data types.

* Shallow copy: A shallow copy copies:

* the top-level array.

* but nested objects/arrays are still shared by reference.

E.x.

`let arr1 = [`

`{ name: "xyz" }`

`];`

creating a shallow copy
spread operator

`let arr2 = [...arr1];`

or we can use

`Array.from(arr1)`

`arr2.name = "ABC";`

`console.log(arr1);` → `[ { name: "ABC" } ]`

`console.log(arr2);` → `[ { name: "ABC" } ]`

* Memory Visualization:

`arr1` → `[ Ref 100 ]`

`arr2` → `[ Ref 100 ]`

Only the outer array was copied.
The object inside is still shared.

Both arrays points to same object.

* Deep copy: A deep copy:

(i) copies the array.

(ii) copies nested objects/arrays too.

`let arr2 = structuredClone(arr1);`

* Symbol: Creates a unique identifier.

Even if two symbols have same description, they are different.
Symbols are mainly used to create: (1) Unique Object Keys
(2) Hidden properties

* Objects: stores data in key-value pairs.

Object literal Syntax:

```
let person = {  
  name: "Dev",  
  age: 21,  
  city: "Pune"  
};
```

Accessing Object Properties:

(1) `console.log(person.name);`
(Dot Notation)

(2) Bracket Notation:

`console.log(person["age"]);`

Bracket notation is useful when:

* Key contains spaces.

* Key is dynamic.

Objects are stored in heap memory.

* this keyword in Objects:

```
let person = {  
  name: "Dev",  
  greet() {  
    console.log(this.name);  
  }  
};  
person.greet();
```

* Some useful object methods:

(1) `Object.keys(person);`

→ `["name", "age", "city"]`

(2) `Object.values(use person);`

→ `["Dev", 21, "Pune"]`

(3) `Object.entries(person);`

[
 ["name", "Dev"],
 ["age", 21]
]

* Object Destruction:

```
let { name: n, age: a, city: c } = person
console.log(n) → "Dev"
console.log(a) → 21
```

* function login (username) {

```
  return ` ${username} just logged in`
}
```

```
console.log(login()) → undefined just logged in
                        ↑
                    without any parameter
```

* Declaring functions:

```
function addOne(num) {
  return num + 1
}
```

(Hoisting) Holding a function inside a variable

```
const addOne = function(num) {
  return num + 2
}
```

* Arrow functions: A shorter way to write functions in Js.

```
function add(a, b) {
  return a + b;
}
```

```
const add = (a, b) => {
  return a + b;
}
```

```
const greet = () => {
  console.log("Hello");
};
```

* this in Normal Vs Arrow functions:

```
const user = {
  name: "XYZ",
  greet: function() {
    console.log(this.name)
  }
};
user.greet();
```

Output: XYZ

Same in arrow function

```
greet: () => {
  console.log(this.name)
}
```

Arrow functions undefined do not create their own this.

- * Arrow functions inherits this from their surrounding.
- * This is called lexical this.
- * this comes from where the function was created.

* How Javascript Executes code:

Javascript executes code in a step-by-step process using:

- (1) Execution context
- (2) call stack
- (3) Memory creation
- (4) Execution Phase

2 Synchronous

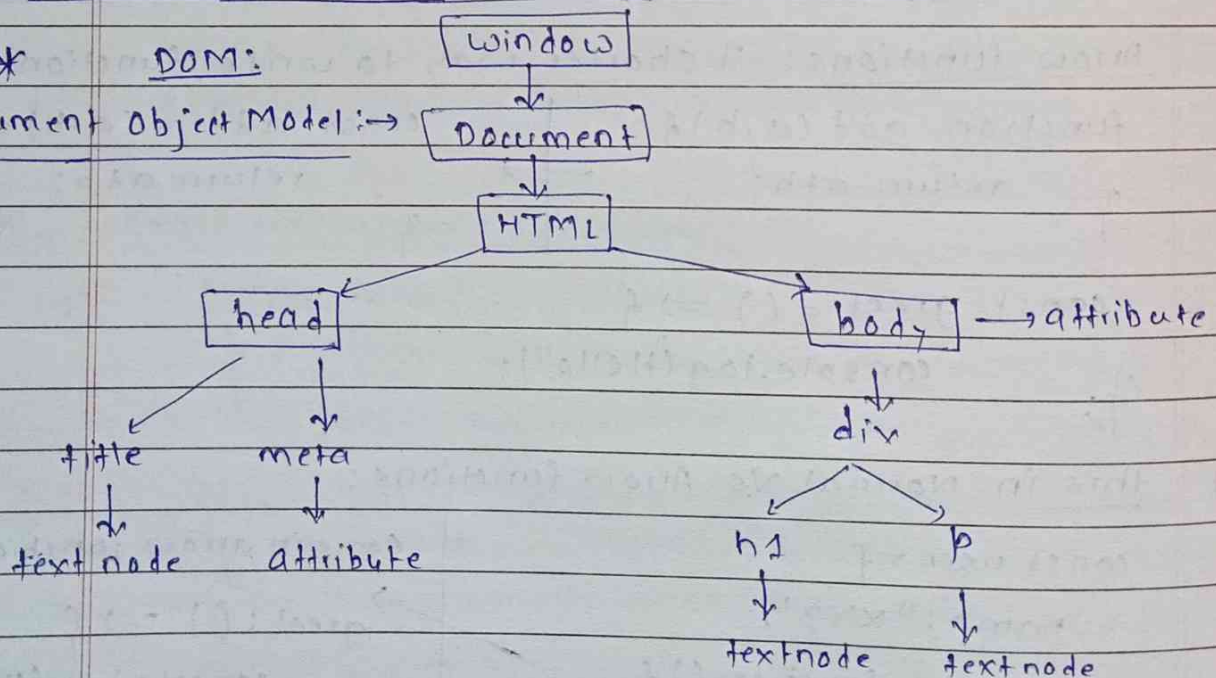
* Javascript is single-threaded: Javascript executes:

- (i) one line at a time
- (ii) In a single call stack

Meaning: One operation executes at a time.

* DOM:

Document Object Model →



* DOM selectors: Used to select HTML elements from a webpage so JavaScript can:

- (i) read them
- (ii) modify them
- (iii) add events
- (iv) change styles

JavaScript converts HTML into a tree-like structure called the DOM.

1. getElementById():

let heading = document.getElementById("title");
 → `<h1 id="title">Hello</h1>`

2. getElementsByClassName(): Selects elements having same class.

let items = document.getElementsByClassName("text");
 → Returns HTML collection.

3. getElementsByTagName():

let paragraph = document.getElementsByTagName("p");
 → returns HTML collection.

4. querySelector(): selects first matching element.

let el = document.querySelector(".text");

5. querySelectorAll()

* HTMLCollection: A collection of HTML elements.

* Array-like but not a real array.

* HTML collection is usually live → It updates automatically when DOM changes.

- * Nodelist → A collection of DOM nodes returned by certain DOM selectors (`querySelectorAll()`)
It is not live.

- * Events in JS: An action that happens in the browser, e.g. clicking a button, typing in input, scrolling, etc. JS can listen to these events & respond.

- * `element.addEventListener(event, callback)`
(click, dblclick, mouseover, mouseout, keydown, keyup)

- * Event Object: Whenever an event occurs, JS automatically passes an event object.

```
button.addEventListener("click", (e) => {
    console.log(e);
});
```

```
<button onclick="alert('Hi')"> Click </button>
```

- * Removing Event Listeners: `removeEventListener("click", greet);`

```
<button> Button 1 </button>
```

```
<button> Button 2 </button> → Clicked → JS automatically creates an event object inside e.target (or event.target) so JS knows exactly which element triggered the event.
```

- * Event Bubbling: When an event happens on a child element, the event automatically moves upward through its parent elements.

```
<div id="parent">
```

```
  <button id="child"> Click </button>
```

```
</div>
```

```
parent.addEventListener("click", () => {
    console.log("Parent clicked")
})
```

When Button Clicked → child clicked
Parent clicked

To stop Bubbling :

```
child.addEventListener("clicked", (e) => {
  e.stopPropagation();
});
```

* Blocking Vs Non-Blocking Code:

(1) Blocking: Stops the execution of the program until the task finishes.

Synchronous code: Code executes line by line, one statement at a time in order.

(2) Non-Blocking: Allows JS to continue executing ~~and~~ other code while waiting for some task to complete.

E.x.

```
console.log("Start");
setTimeout(() => {
  console.log("Inside Timeout");
}, 3000);
console.log("End");
```

} Start
End
Inside Timeout

* Asynchronous Javascript: Start a task and continue executing other code without waiting for that task to finish.

Why async is needed:

(i) Javascript is single-threaded - only one thing runs at a time.

(ii) If long tasks blocked execution:

* UI would freeze

* website become slow.

* callback Functions: A function passed as an argument to another function, which is executed later.

E.x.

```
function greet(name) { . . . . . }
function processUser (callback) {
  let name = "xyz";
  callback (name);
}
```

```
processUser (greet);
```

Modern E.x.

```
setTimeout ( () => {
  console.log ("xyz");
}, 2000);
```

call back function

```
button.addEventListener ("click",
() => {
. . . . .
});
```

* JSON.stringify():

JavaScript Object → JSON string

```
let user = {
  name: "Dev",
  age: 21
};
```

- * APIs send text.
- * localStorage stores strings
- * network communication uses strings

```
let jsonData = JSON.stringify (user);
console.log (jsonData);
```

↓
" { "name": "Dev", "age": 21 } "
Now it is a string

* JSON.parse(): JSON string → Javascript Object

Sending Data:

Receiving Data:

```
fetch ("/api/", {
  method: "Post",
  body: JSON.stringify (user)
});
```

```
let data = await response.json();
```

* Promise: A promise is an object that represents the eventual completion or failure of an asynchronous operation.
why Promises were introduced: Before promises, JS used: callbacks:

```
login ( ) => {
  getUser ( ) => {
    getPosts ( ) => {
      ...
    }
  }
}
```

} This is called callback Hell.

also called executor

* creating Promise:

```
const promiseOne = new Promise ( function (resolve, reject)
=> {
  // Do any async task, DB task, etc.
  setTimeout ( function () {
    console.log ( ... ); resolve ( )
  }, 2000)
});
```

consuming promise:

```
promiseOne.then ( function () {
  ...
})
```

* The Three states of a Promise:

A promise is always in one of these three states.

(i) Pending: The initial state, the operation hasn't finished yet.

(ii) Fulfilled (Resolved): The operation completed successfully and the Promise now has a value.

(iii) Rejected: The operation failed & the Promise has a reason (error) for the failure.

creating a promise using
new Promise constructor

```
const myPromise = new Promise ( (resolve, reject) => {  
  const success = true;  
  setTimeout ( () => {  
    if (success) { resolve ( " . . . . . " ); }  
    else { reject ( " . . . . . " ); }  
  }, 2000 );  
});
```

↑ this funⁿ runs immediately & receives two arg. resolve & reject.

* once a promise is created, you need to handle the result when it moves out of the "Pending" state.

Using :

- then() → Runs when the promise is fulfilled.
- catch() → Runs if the promise is rejected.
- finally() → Runs regardless of the outcome.

myPromise .

```
.then ( (data) => { console.log (data); } )  
.catch ( (error) => { console.log (error); } )  
.finally ( () => { console.log ( "Op. finished" ); } );
```

* Promises solve the problem of callback Hell:

Because .then() always returns a new Promise, you can chain operations vertically rather than nesting them.

* When we initialize a new Promise(), you provide an executor function. The function runs immediately.

Inside this function, you have two 'values':

- (i) resolve(value): Pushes data into the success track.
- (ii) reject(error): Pushes data into the error track.

* Once the executor calls one of those values, the data flows into specific "handlers":

* If `resolve()` is called:

- (i) The data is wrapped in a fulfilled Promise.
- (ii) It travels to the first `.then()` block.
- (iii) If that `.then()` returns a value, the value is wrapped in a new Promise & passed to the next `.then()` in the chain.

* If `reject()` is called:

The data skips all `.then()` block until it finds `.catch()` block.

* async - await:

```
async function consumePromise () {
  const response = await promiseFive
}
```

When we place `async` before a function declaration, you are telling JS two things:

- (1) The function will always return a Promise. Even if you return a simple string, JS wraps it in a resolved Promise.
- (2) It allows the use of `await` keyword inside the function.

await: This can only be used inside an `async` function. It tells JS - "pause the execution of this special function until the Promise is settled (resolved or reject).

* E.x. You need to fetch a user, then fetch their posts & finally fetch comments for the first post.

* Using Promise chaining:

```
getUser(1)
```

- then (user $\Rightarrow$ getPosts(user.id))
- then (posts $\Rightarrow$ getComments (posts[0].id))
- then (comments $\Rightarrow$ console.log (comments))
- catch (err $\Rightarrow$ console.log (err));

* Using Async / await

```
async function displayComments () {
```

```
  try {
```

```
    const users = await getUsers (1);
```

```
    const posts = await getPosts (user.id);
```

```
    const comments = await getComments (posts[0].id);
```

```
  }
```

```
  catch (err) { console.error(err); }
```

```
}
```

← The rest of the script keeps running.

Inside async function: The code won't move to the next line until the await finishes.

```
async function getAllUsers () {
```

```
  try {
```

```
    const response = await fetch ("https://.....")
```

```
    const data = await response.json()
```

```
    console.log (data);
```

```
  } catch (err) {
```

```
    console.log (err);
```

```
  }
```

```
}
```

Using fetch

with async

await:

Parsing also takes some time

* npm → Node Package Manager

It is the default package manager for Node.js

- (i) Downloads packages from the npm registry
- (ii) stores them in node_modules/
- (iii) saves dependency info in package.json.

* npx → node package Execute : A tool that comes with npm.

It is used to run packages directly without installing them globally.

npx create-react-app my-app

- (i) It downloads create-react-app temporarily.
- (ii) Executes it.
- (iii) Doesn't permanently install it on your system.

(1) dependencies: These are packages required for your app to run in production.

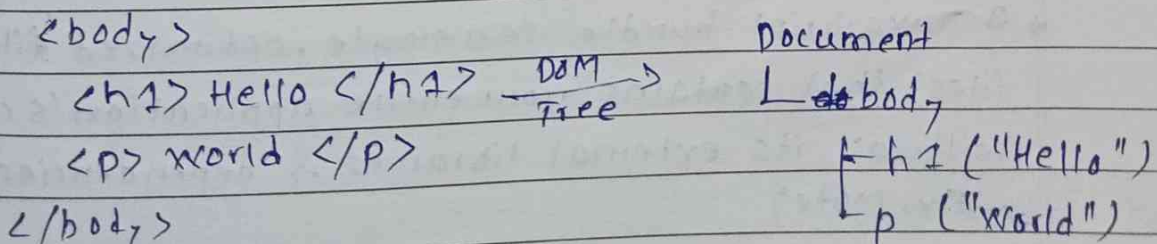
(2) devDependencies: Packages needed only during development. (E.x. Nodemon, prettier, etc.)

(3) scripts: shortcuts to run commands.

(4) eslintConfig: Finds bugs early, maintains consistency.

* DOM (Document Object Model):

A programming interface that represents the HTML page as a tree like structure of objects.



Each element becomes a node that JavaScript can access and manipulate.

* `package.json` → main project file where you define:
Project metadata (name, version, description),
Dependencies, scripts, etc.

* Versions are usually flexible.

^18.2.0 → Allows installing 18.x.x

It describes requirements, not exact installs.

* `package-lock.json` -

(i) Exact versions of every installed package.

(ii) Dependency tree

(iii) Resolved URLs & integrity hashes.

`"react": {
 "version": "18.2.0"
}` } ⇒ npm install ⇒ Always installs 18.2.0 version even though new versions are available.

* `index.html` → Entry point of web app.

(i) It's the first file the browser loads.

(ii) It provides the root container where your app is rendered.

(iii) It links the JavaScript bundle (React, Angular)

* A JavaScript bundle is a single, optimized file (or set of files) that contains your entire application's code along with all its external libraries & dependencies.

Ex. (vite)

* `main.jsx` → It is the entry point of React app (JS-side)
 connect to react with DOM
`ReactDOM.createRoot (document.getElementById ("root")).`

`render (`
`< React.StrictMode >`
`< APP />`
`< /React.StrictMode >`
`);`

Takes control of the DOM

connects to index.html

Development-only tool.
 (It does not affect production)
 & helps you catch problems early.

* `index.html` → `main.jsx` → `App.jsx` → other components.

* `function App () {`
`return (`
`<h1>hello</h1>`
`<p>World</p>`
`);`
`}`

A component must return one root node.
 Not allowed

correct way:

`<div>`
`<h1> ... </h1>`
`<p> ... </p>`
`</div>`

why? →

React builds something called a virtual DOM tree.

For react, to: (i) compare changes efficiently.
 (ii) Render updates properly.
 , It needs a single root node per component.

* Fragments (better way):

`function App () {`
`return (`
`<>`
`<h1></h1>`
`<p>World</p>`
`</>`
`);`

Another way:
 Use `<React.Fragment>`
 instead of `<></>`
 Fragments (No extra DOM element is created)

* Hooks in React: Hooks are special functions that let you use state & other React features inside the functional components.

E.x.

(1) Counter using pure javascript:

```
<!DOCTYPE html>
<html>
<body>
  <h1 id="count">0 </h1>
  <button id="inc">+ </button>
  <button id="dec">- </button>

  <script>
    let counter = 0;
    const countEl = document.getElementById("count");
    const incBtn = document.getElementById("inc");
    const decBtn = document.getElementById("dec");

    incBtn.addEventListener("click", () => {
      counter++;
      countEl.innerText = counter;
    });
  </script>
</body>
</html>
```

Manually select DOM elements.

Handle UI yourself & events directly

If counter is used at many other places, we have to

- (i) Manually select DOM element
- (ii) Handle UI that no. of times

(2) Counter using React Hooks: React controls UI updation

(i) function App() {
 use Let ← let [count, setCount] = useState(0);
 ← Current value ← Function to update it.
 (const) ← Gives error as it cannot be reassigned
 return (
 <>
 <h1> count </h1>
 <button onClick = { () => setCount (++count) } > Add </button>
 } > Add </button> } Event Handling
 </>
);
 }
 }

(ii) const [count, setCount] = useState(0);
~~const~~ increment = () => {
 if (count < 100) { setCount (count + 1); }
 };
 const decrement = () => {
 if (count > 0) { setCount (count - 1); }
 };
 return (
 <>
 <h1> count </h1>
 <button onClick = { increment } > + </button>
 <button onClick = { decrement } > - </button>
 </>
);
 }
 }

* Virtual DOM: A lightweight javascript representation of the real browser DOM.

Directly updating the real DOM is:

* Slow * Expensive

So React uses an in-memory copy (Virtual DOM) to make updates efficiently.

Real DOM:

```
<div>
  <h1> count: 1 </h1>
</div>
```

Virtual DOM:

```
{ type: "div",
  children: [
    { type: "h1",
      text: "count: 1"
    }
  ]
}
```

React work with this JS object instead of touching the real DOM directly.

State changes (setCount) → React creates a new Virtual DOM → Compares it with prev VDOM → Updates only the changed part in real DOM.

* Reconciliation: The process of comparing old VDOM and new VDOM & updating the real DOM efficiently

Before click:

```
<h1> count : 2 </h1>
```

After click:

```
<h1> count : 3 </h1>
```

React detects → only text changed

No need to re-render whole <div>

So it updates only : textContent

* This process is called Diffing: (diffing algorithm) →

(i) compares trees

(ii) Find minimal changes.

* Diffing: Comparing the old VDOM & new VDOM to find the minimum changes needed for the Real DOM. This is a part of reconciliation.

Ex. Before:

<div>

<h1> Title </h1>

<p> Text </p>

</div>

After:

<div>

<h1> Title updated </h1>

<h1> Text </h1>

</div>

Instead of checking everything deeply (slow),
Diffing: React uses heuristics (rules) to make it faster.

<div> → same → go inside

<h1> → same → update text only

<p> != <h1> → Removes <p> & create <h1>

* React Fibre: A smarter reconciliation engine that breaks rendering work into smaller units & schedules them efficiently.

Before Fibre:

* Rendering was synchronous

* Once started → React could not pause.

* Large updates → UI could freeze

Fibre makes rendering → Interruptible, Prioritized.

Priority level: →

High priority → user input

Low priority → background updates.

- (inputs to a component)
- * props: (Properties) are used to pass data from one component to another.
props are immutable.

E.x.

(1) Parent component

```
function App() {
  return (
    <>
    <Greeting name="xyz"/>
  </>
);
```

(2) child component:

```
function Greeting(props) {
  return <h1> Hello,
    {props.name} </h1>;
}
```

```
function Card({children}) {
  return <div> {children} </div>;
}
```

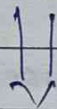
```
<Card>
  h1 <Hello </h1>
</Card>
```

* Props help build reusable components.

* Enable data flow.

* Functional updates:

```
const [counter, setCounter] = useState(0);
<button onClick = { () =>
  setCounter(counter + 1);
  setCounter(counter + 1);
  setCounter(counter + 1);
} />
```



It will give 1 as answer not 3

why —> * Because count does not update immediately.

* React queues updates and applies them later.

* So each call still sees the old value 4.

* Functional components: Simple Javascript functions that accept 'props' as an argument and return a React element.

Page No.	
Date	

* Correct way to get 8 → Functional updates

setCount (prev ⇒ prev + 1);	0 → 1
setCount (prev ⇒ prev + 1);	1 → 2
setCount (prev ⇒ prev + 1);	2 → 3

Each update gets the latest state, not the stale one.

* If next step depends on previous state → always use functional update.

* React Component Lifecycle: In react, lifecycle besides - how a component is created, updated & removed from DOM.

↳ Main Phases: Mounting → Updating → Unmounting

① Mounting Phase: Component is created.

When component appears on screen for the first time.

Flow (Functional components):

E.x.

```
import { useEffect } from "react";
```

```
function App() {
```

```
  useEffect ( () ⇒ {
```

```
    console.log ("Mounted!");
```

```
  }, [] );
```

```
  return <h1> Hello </h1>;
```

(i)

Function runs

→ JSX returns

(ii) Virtual DOM created.

(iii) Real DOM updated.

(iv) useEffect runs

② Updating Phase:

* Happens when state changes (useState)

* props changes.

```
useEffect ( () ⇒ {
```

```
  console.log ("Updated!");
```

```
  }, [count] );
```

State change

↓
Re-Render

↓
Diffing

↓
DOM updates

↓
useEffect

3

Unmounting Phase: when component is removed from DOM
* cleanup function runs before component is destroyed.

* `import { useEffect } from "react";`

↑ (named export)

Javascript ES module syntax

React exports things in two ways:

(1) Default exports

`import React from "react";`

(2) Named exports

`import { useEffect, useState } from "react";`

These are specific functions exported by name.

* `useEffect(() => {
 console.log();`

`}, [ ] );`

↑

Dependency array $\Rightarrow$ tells react, when should this effect run.

Different cases:

(1) Empty ~~as~~ array `[]`:

Runs only once (after first render)

(2) with dependencies: `[count]`

Runs: (1) First render (2) Every time count changes

(3) Case 3: No array $\rightarrow$ Runs on every render

* useEffect : Used to run side effects in functional components.
side Effects → Anything that happens outside rendering UI. e.g.

- (i) Fetching data from an API
- (ii) Adding event listeners.
- (iii) Updating the DOM manually.

```
useEffect ( () => {
  // side effect logic
  return () => {
  };
}, [dependencies])
```

* useEffect runs after the component renders.

(1) Runs on every Render:

```
useEffect ( () => {
  console.log ( " " );
});
```

← No dependency array

} Runs after every render.
Initial render + every update

(2) Runs only once (on mount):

```
useEffect ( () => {
  console.log ( " " );
}, [ ] );
```

(3) Runs when dependencies changes:

```
useEffect ( () => {
  // ...
}, [count] );
```

Render UI → useEffect runs (after render) → If state changes inside useEffect → Re-render happens.

* useEffect re-runs after re-renders only if its dependencies have changed.

Without dependencies, it runs on every render, which can cause infinite loops if state is updated inside it.

- * useRef: Lets us store a value that:
 - (i) persists across renders.
 - (ii) does not cause re-render when it changes.

```
function App () {
  const countRef = useRef(0);
  const handleClick = () => {
    countRef.current += 1;
  };
  return (
    <>
    <button onClick={handleClick}> Click </button>
    </>
  );
}
```

countRef is an object
{ current: 0 }

* clicking button updates
countRef.current
* Value changes.
* UI does not re-render.

- * Accessing DOM: const inputRef = useRef();
const AccessDOM = () => {
 inputRef.current.focus();
}
- Points to actual DOM node

- * Store previous value:

```
function App () {
  const [count, setCount] = useState(0);
  const prevCount = useRef();
  useEffect(() => {
    prevCount.current = count;
  }, [count]);
  return (
    <>
    <h1> Current: {count} , Previous: {prevCount.current}
    </h1>
    <button onClick={() => setCount(count + 1)}> ...
  );
}
```

Use it for: (i) Accessing DOM elements.
(ii) Storing mutable values (timers, IDs, previous values)
(iii) Avoid re-renders for non-UI data.

* useCallback: It remembers (memoizes) a function so React doesn't create a new one on every render.

In React:

- (1) Every render → new function is created from scratch.
- (2) New function = new reference.
- (3) That can cause unnecessary re-renders.

without useCallback():

```
const sayHello = () => {  
  . . . . .  
};
```

Say Hello Button

* Click Say Hello Button

* Component re-renders

* sayHello is created every time.

* New function every render

with useCallback():

```
const sayHello = useCallback(()  
=> {  
  . . . . .  
}, []);
```

* First render → function is created.

* Click sayHello button → re-render happens.

* But function is not created (same function reference is used)

* Same function reference unless dependencies changes.

useCallback(function, [dependencies])

* If dependencies change → function recreated

* If not → same function reused.

* useMemo: useMemo is the sibling of useCallback, but instead of memoizing a function, it memoizes a computed value.

(i) It caches the result of calculation.

(ii) Recomputes only when dependency changes.

without useMemo:

* Every render $\rightarrow$ all calculations run again.

* If a calculation is heavy $\rightarrow$ performance issues.

with useMemo:

```
const result = useMemo(() => {
  return count * 2000;
}, [count]);
```

* First render $\rightarrow$ calculation runs

* count changes $\rightarrow$ recalculations

* If something else cause re-render $\rightarrow$ no calculation again

* useContext: Helps to share data across components without passing props manually at every level (no prop drilling)

App $\rightarrow$ Parent $\rightarrow$ child $\rightarrow$ Grandchild

If Grandchild needs some data (like user info), we normally have to pass it through every level as props

* useContext lets any component directly access shared data.

(Avoid when: Data changes very frequently (can cause many re-renders))
E.x. theme

Context:

export const ThemeContext = createContext();

Consumer:

```
const theme = useContext(ThemeContext);

return (div className = {theme}) Hello </div>
```

Provider:

```
<ThemeContext.Provider value="dark">
  <Navbar />
```

```
</ThemeContext.Provider>
```

* when context value changes, all components using it re-renders

*

creating custom hook - useAuth :

- * Name must start with use
- * can use other hooks inside.
- * Must be called inside React components (not normal func.)

(1) create Auth Context: This acts as a global container for the authentication state so we don't have to pass props down manually through every component.

creates the context

```
import React, { createContext, useState, useEffect } from 'react';
```

```
const AuthContext = createContext(null);
```

↑ useContext
↑ Uses (reads) the context

(2) create the Auth Provider: The Provider is a component that will wrap your application. It holds the actual state (like the user object) and functions to change the states.

```
export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [userProfile, setUserProfile] = useState(null);
```

```
  const login() { }
```

```
  const logout() { }
```

```
  const signin() { }
```

```
  const signout() { }
```

```
  const value = {
    user, isAuthenticated, login, logout
  }; // Obj containing everything we want to share via our hook
```

```
  return (
```

```
    <AuthContext.Provider value={value}>
```

```
      {children}
```

```
    </AuthContext.Provider>
  );
```

(3) In App.js, wrap your component tree in the AuthProvider

```
import {AuthProvider} from './AuthContext';  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(  
  <AuthProvider>  
    <App/>  
  </AuthProvider>  
)
```

* React Router: ~~npm~~ npm install react-router-dom

React is designed to build Single Page Applications (SPA)

* Traditional UI Rendering: Every time we navigate:

- (1) Browser sends request to server.
- (2) Server returns a new HTML page.
- (3) Entire page reloads.

* Single Page Application: A web app that loads a single HTML page once and then updates the UI dynamically using Javascript, without reloading the entire page.

(1) React Router watches the URL in the browser's address bar.

(2) When the URL changes, it tells React exactly which components to show, all without ever refreshing the page.

* 3 core components of React Router:

(1) <BrowserRouter> → This wraps your entire app. It connects your React app to the browser's URL history.

- (2) `<Routes>` → Acts as a container for all individual routes.
- * Whenever URL changes, `<Routes>` looks through all of its children (`<Route>` components) to find the best match for the specific URL.
 - * Once it finds the winner, it renders that component & ignores the rest.

- (3) `<Route>` → Defines a single rule connecting a URL path to a specific React component.

```
function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
```

* ProtectedRoute: A special component that checks if a user is logged in before letting them see a specific page.

```
      <Route path="/" element={ <Home/> } />
```

```
      <Route path="/login" element={
```

```
        <LoginPage/> } />
```

```
    </Routes>
```

```
  </BrowserRouter>
```

```
  </AuthProvider>
```

```
};
```

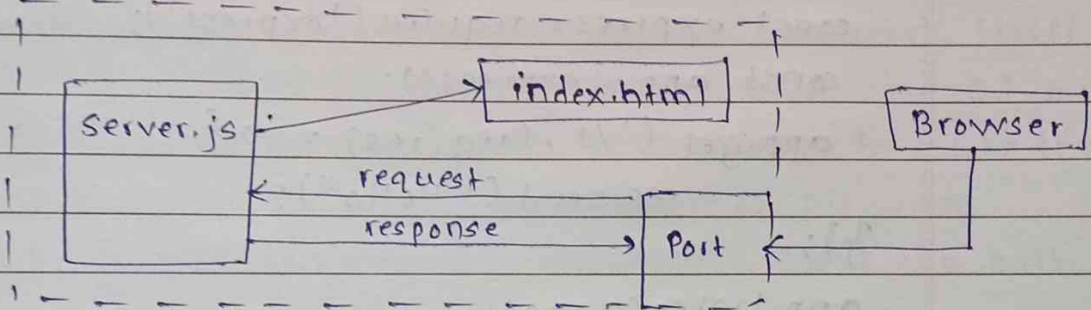
Backend

Page No.	
Date	

- * NGINX: A high-performance web server mainly used for:
 - (i) Serving websites & static files.
 - (ii) Forward requests to backend applications.It is extremely popular for hosting applications built with Node.js, Python, Java.

client Browser → NGINX → Node.js APP

- (iii) Used mostly in production servers, cloud deployments, VPS hosting.



- * creating a server:

```
const port = 3000;
const server = http.createServer();
server.listen(port, () => {
  // ...
});
```

- * Two major components of any application:
 - (1) Routing structure
 - (2) Database

↓

Express

- * Express: A web framework built on top of Node.js. It helps developers build:
 - * Web servers
 - * APIs
 - * Backend applicationsmore easily & with less code.

* Ex. using pure Node.js to create a server:

```
const http = require('http');  
const server = http.createServer((req, res) => {  
  res.end("Hello");  
});  
server.listen(3000);
```

Writing large applications this way becomes difficult

* Express simplifies backend development.

```
const express = require('express');  
const app = express();  
app.get('/', (req, res) => {  
  res.send("Hello");  
});  
app.listen(3000);
```

APP → Express.js → Node.js → OS

* Main features of Express:

(1) Routing: Easy URL handling.

```
app.get('/about', (req, res) => {  
  // ...  
});
```

(2) Middleware: Functions that process requests before response.
Ex. Auth, Logging, Validation

(3) API creation: Widely used for REST APIs.

(4) Easier request/response handling:

Express provides helper methods like:

* res.send(), res.json(), req.params, req.body

Dynamic segments

```
app.get('/users/:userId', ...)  
req.params => { userId: '234' }
```

* CORS (Cross Origin Resource Sharing):
It is a browser security mechanism that controls whether one website can access resources from another website.

* Origin: Protocol + Domain + Port

E.x. https://example.com

↑ ↑ Port = 80
Protocol domain

* Port: A virtual communication endpoint that allows a device to separate different types of network traffic.
(i) When a data packet arrives at a device, the OS uses port no. (which range from 0 to 65535) to route the traffic to appropriate software ~~engine~~ program.

(ii) E.x. A web browser and an email client can both be open on the same computer because they use different ports to handle their respective incoming data.

(iii) Specific ports are reserved for specific internet services:

Port 80: HTTP

Port 22: SSH

Port 443: HTTPS

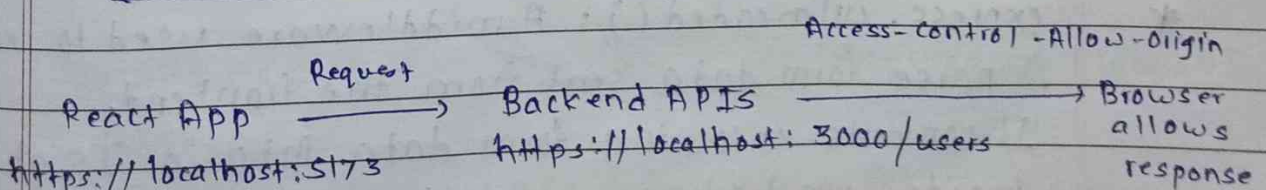
Port 53: Used for DNS

* Why CORS exists: Imagine logging into a bank website: If any malicious website tries to secretly access bank account data using JS.

∴ Browser enforces → Same-Origin Policies

Meaning: A webpage can normally only access resources from the same origin.

CORS is the controlled way to allow exceptions.



* common CORS Headers:

(1) Allow origin:

Access-Control-Allow-Origin: *

OR

Access-Control-Allow-Origin: http://localhost:5173

Allow all origins

(2) Allow Method:

Access-Control-Allow-Methods: GET, POST

(3) Allow Headers:

Access-Control-Allow-Headers: Content-Type

```
app.use ( cors ( {  
  origin: ' . . . . .',  
  methods: [ "GET", "POST", . . . ],  
  allowedHeaders: [  
    "Content-Type",  
    "Authorization"  
  ]  
} ) ) ;
```

* Preflight Requests: For complex requests, browser first sends: https: OPTIONS request before actual request.

To check whether server permits requests.

* CORS is enforced by: Browsers

Hence tools like Postman, curl usually ignores CORS.

* express.urlencoded(): A middleware used to read & parse form data sent from the frontend. It converts incoming form data into a JS object inside req.body

If form submits : username = xyz , age = 23

This form data is encoded like URL query parameters:
https://localhost:8000/user/name=John&age=20

application / x-www-form-urlencoded

app.use(express.urlencoded({ extended: true }));

∴ Now req.body = {

username = "dev",

age = "23"

↑
supports nested
objects

* express.json() → JSON request body

express.urlencoded → HTML form data

* express.json(): Parse incoming JSON data from requests & store it as object.

Frontend sends: { name: "Dev" } → This data is sent as a raw text stream inside the HTTP request body.

express.json() →

(i) Reads incoming request body.

(ii) Converts raw JSON text into JS object using JSON.parse()

(iii) Stores it inside req.body

(iv) Without it, console.log(req.body) → undefined

* Mongoose: Library for Node.js that helps you work with MongoDB more easily.

It acts as a bridge between Node/Express & MongoDB database.

* Mongoose is an ODM for MongoDB

* ODM (Object Data Modelling): It is a technique that maps Database documents $\longleftrightarrow$ Programming language objects. ODM helps by providing: schemas, Validation, Easy database operations.

* Without mongoose:

```
db.connection("users").insertOne({  
  name: "Dev",  
  age: 20  
});
```

Schema defines the structure of documents.

* With Mongoose:

```
const userSchema = new mongoose.Schema({  
  name: String,  
  age: Number  
});
```

Automatically provides created at & updated at

* Then create Model:

```
const User = mongoose.model("User", userSchema);
```

Javascript class created from schema

* Usage: `User.create({ name: ... });`

* Advantages:

(1) Schema Validation: `age: Number`

If someone sends: `age = "hello"`

mongoose can reject it.

(2) Middleware / Hooks:

```
userSchema.pre("save", function() {  
  // ...  
});
```

Runs before saving document.

(3) Can connect collections using references:

```
schoolId: {  
  type: mongoose.Schema.Types.ObjectId,  
  ref: "School",  
}
```

* next → To proceed to other hook once task is done

* Not to use an arrow function here & array function do not provides 'this'.

```
userSchema.pre("save", async function(next) {  
  this.password => await bcrypt.hash(this.password  
  next()  
})
```

compare(—, —) for checking password

This pre-hook will run for all the 'save' operations that will be done
To stop this behaviour

```
if (!this.isModified("password"))  
  return next()
```

* JSON Web Token (JWT): It is an open standard that defines a compact & self-contained way for securely transmitting information between parties as JSON object.
* It information can be trusted because it is digitally signed.

* When JWT should be used:

(1) Authorization (2) Information Exchange

* Authorization: Bearer <token> → syntax

* JWT structure: Header + Payload + Signature
xxxxxx.yyyyyy.zzzzzz

(i) Header: { "algorithm": "HS256",
"type": "JWT"
}

(ii) Payload: { "id": "123456",
"name": "xyz",
"role": "admin"
}

This is called a claim
→ Registered
→ Public
→ Private

This payload is the Base64 url encoded.

(3) Signature:

SHA256(
base64url(header) + base64url(payload)
) + secret key

* Access Token & Refresh Token:

Used in authentication systems to securely keep users logged in.

* Why Two tokens: → If only one long-lasting token exists, stealing it means getting full account access for longer time.

(Storing this one password in http cookie can be a safe option but still it will be vulnerable to other types of attack).

(1) Access token: A short-lived token used to access protected resources. (5 min - 1 hour)

(2) Refresh token: A long-lived token used to generate new access tokens. (7 days - 1 month)

* Authentication Flow:

Step 1: login → User sends: email + password
Server verifies users.

After verification, server creates: Access & Refresh token

Step 2: Tokens are send to client

Access token → stored in localStorage / session storage
Refresh token → Secure HTTP-only cookie

JavaScript cannot access it
safer from XSS attacks.

Because used frequently

Step 3: Frontend sends token with every API request.
 Authorization: Bearer <token>

Server verified it $\xrightarrow{\text{valid}}$ Allow access
 $\searrow$ 401 unauthorized

Step 4: When access token expires:

Front sends refresh token to server.

If valid: Server creates new access token & sends it back.
 $\therefore$ Users stay logged in without re-entering password.

* If refresh token is expired mid way after request before reaching response?

→ (i) Server returns → 401 unauthorized or token expired

(ii) Client usually have logic like:

If 401 due to token expiry:

Use refresh token → Get new access token →

Retry original request

* Real applications handles this gracefully → Libraries like

* Axios Interceptors

* Firebase Auth

* OAuth

* Firebase Handles Access & Refresh tokens internally

Frontend App → Firebase Auth SDK → Firebase Server

(i) While using Firebase, we usually only interact with:
 * currentUser * getIdToken()

(ii) login & registration are usually done directly from frontend.

Backend verifies: admin.auth().verifyIdToken(token)
 (Frontend & Backend both contains Firebase credentials of the user independently.)

* Multer: A middleware in express.js used to handle files.
→ images, PDFs, video, documents

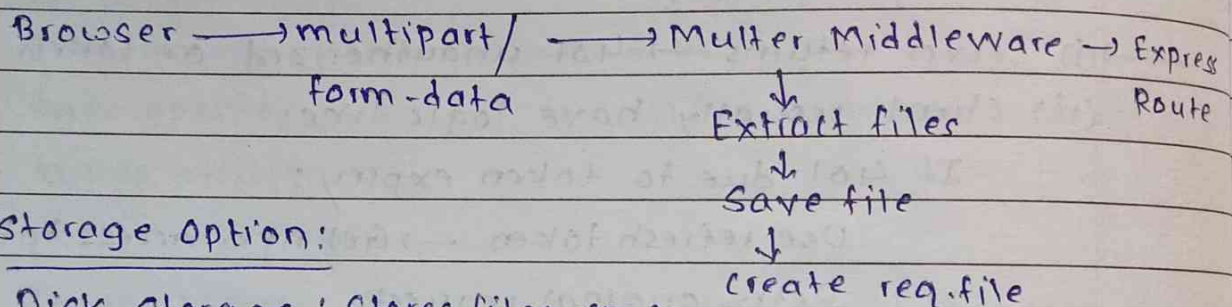
* File uploads uses format → multipart/form-data

Express cannot parse this format by itself.

∴ Multer parses uploaded files & makes them available in req.file & req.files.

* Browser sends request like - multipart/form-data which contain:

(i) file binary data (ii) form-fields (iii) metadata



* Storage Option:

1) Disk Storage: Stores files on server (uploads/)

2) Memory Storage: Stores files in RAM buffer.
Useful for: cloud uploads & image processing.

* File Validation: Multer can validate:

* file type

* size

* count

* How file is sent from client to server:

* Files are binary data.

* JSON is text-based & not suitable for directly sending files efficiently.

* The uploaded file is sent as:

* raw binary bytes, not Base64, not JSON.

* Content-Type: Content-type is an HTTP header that tells what kind of data is being sent.

(*) Backend need to know how to parse incoming data.

(1) application/json:

```
{ "name": "Dev",  
  "age": 21  
}
```

Used with: express.json()

Frontend object → JSON.stringify() → JSON text → Backend
parse JSON

(2) application/x-www-form-urlencoded: Traditional HTML form-data.

Body: name=Dev&age=20

Used with: express.urlencoded()

(3) multipart/form-data

(4) text/plain: Plain text.

(5) text/html: HTML pages

(6) image/png, image/jpeg: Used for image responses.

(7) application/pdf: Used for PDFs. Browser opens/download PDF.

(8) application/octet-stream: Generic binary data.

* HTTP request Methods: Defines what action the client wants to perform on the server.

(1) Get Method: retrieve data

* Data usually sent via URL / query params

* can be cached.

* POST: create / send new data

* changes server state.

* PUT: replace / update entire resource / object

* * Usually idempotent $\rightarrow$ Same request gives same final result.

* ~~used~~ PATCH: partially update resource

Only updates specified fields.

* DELETE: Delete resources.

* OPTIONS: Used to ask server what HTTP methods are allowed.

Very important in: CORS & preflight requests

E.x. OPTIONS /users

Server responds: $\rightarrow$ Allow: GET, POST

* HEAD: Similar to GET but:

* returns only headers.

* no response body

Used for: * checking file existence

* metadata

* caching info

* Safe Vs Unstate methods:

Safe methods: Do not modify data: GET, HEAD

Unsafe methods: Modify server state:

POST, PUT, PATCH, DELETE

* URL & Query params:

In a GET request, data is usually sent through the URL itself instead of the request body.

Two common ways:

(i) URL Parameters (Route Params):

These are values embedded directly inside the URL.

E.x. `/users/15` (`/users/:id`)

↑
`req.params.id = 15`

(ii) Query Parameters: These are key-value pairs added after `?` in URL.

E.x. `/users?page=2&limit=5` (`'/users'`)

Here `page=2`, `limit=5` → query parameters.

* Commonly used in filtering, searching, sorting, etc.

URL Params

`/users/15`

↑
specific resource

Query Params

`/users?page=2`

↑
extra options/filtering

* Security: Neither URL params nor query params are 'safer' because both are part of URL.

* Both are visible in browser history.

* logged by server.

* visible in network logs.

URL Params

`/user/15`

Query Params

`/user?id=15`

⇌ Exposed ⇌

Safer Alternative: sensitive data should usually go in request body (POST, PUT)

* headers (Authorization)